

JobPilot: AI-Powered Job Matching Copilot

BAX-423 Final Project Technical Brief

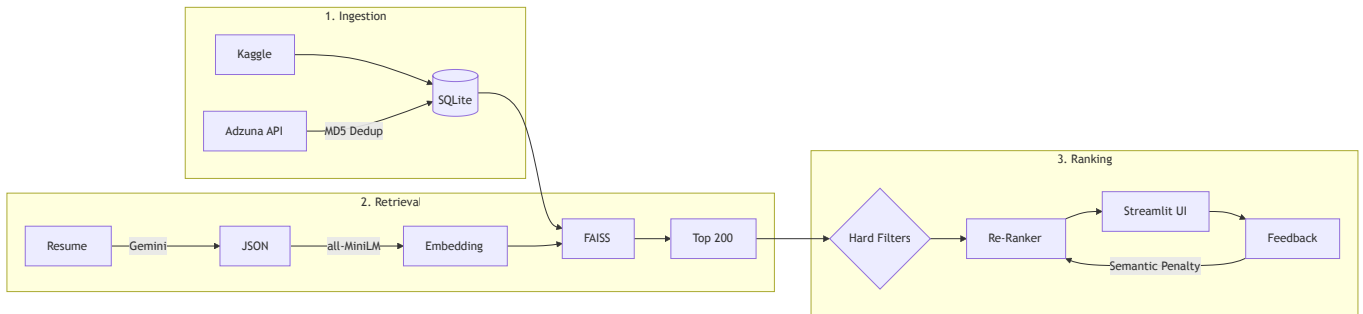
1. Executive Summary

JobPilot is an end-to-end, adaptive job recommendation engine designed to bridge the semantic gap between a candidate's unstructured resume and complex job descriptions. Moving beyond traditional keyword-based applicant tracking systems (ATS), JobPilot leverages state-of-the-art dense vector embeddings, a robust multi-stage ranking pipeline, and implicit user feedback to deliver highly personalized, real-time job matches.

This document provides a comprehensive technical overview of the system's architecture, underlying BAX-423 technique choices with explicit mathematical justifications, offline benchmark evaluations across multiple edge-case personas, database design, and current system limitations with proposed architectural improvements.

2. System Architecture & Pipeline Design

JobPilot follows a highly modular, multi-tier architecture capable of handling both offline batch data and real-time API streams. The system is designed to scale and provide sub-second retrieval times while executing complex ranking logic.



2.1 Hybrid Data Ingestion & Storage Pipeline

To solve the cold-start problem while maintaining real-world utility, the pipeline utilizes a dual-ingestion strategy:

- 1. **Offline Baseline Ingestion:** The system is seeded with over 30,000 structured LinkedIn job postings. This data provides the foundational vector space for our embedding models.
- 2. **Real-Time Streaming Engine:** The system integrates with the Adzuna API using Python's `queue.Queue` and daemon threading. A producer thread fetches raw JSON payloads, while a consumer thread applies an MD5 hashing algorithm to the job descriptions to drop duplicates in real-time before writing to the SQLite database.

2.2 Profile Processing via LLM

Raw PDF resumes are notoriously difficult to parse due to varied formatting. Resumes are processed using Google's **Gemini 2.5 Flash** API with a strict JSON schema enforcement prompt. The LLM extracts unstructured text into structured arrays containing the user's explicit technical skills, soft skills, target job titles, and hard constraints (e.g., minimum acceptable salary, visa sponsorship necessity, preferred locations). This structured JSON is the cornerstone of the downstream hard-filtering stage.

3. BAX-423 Technique Choices & Implementations

3.1 Dense Embedding Retrieval vs. Sparse (Lecture 5)

Traditional applicant tracking systems rely heavily on BM25 (TF-IDF) sparse keyword retrieval. However, job titles and skills suffer from extreme vocabulary mismatch. For example, a candidate whose resume highlights "Data Visualization, SQL, and Python" might be perfectly suited for an "Analytics Engineer" role, yet a BM25 system will rank this candidate poorly if the exact keywords do not match.

- **Technique Choice:** We implemented Dense Embedding Retrieval using HuggingFace's **sentence-transformers/all-MiniLM-L6-v2**. This transformer model maps both resumes and job descriptions into a shared 384-dimensional dense vector space, allowing the system to understand semantic intent rather than just lexical overlap.
- **Vector Engine Integration:** We utilize **FAISS** (Facebook AI Similarity Search) with an **IndexFlatL2** index to perform highly efficient nearest-neighbor searches. This allows us to reduce our 30,000+ record database to 200 highly semantic candidate matches in under 50 milliseconds.

3.2 Multi-Stage Ranking Pipeline (Lecture 7)

While dense embeddings are excellent for semantic recall, they are mathematically incapable of enforcing discrete business rules. A vector space cannot deduce that a \$50k salary is an absolute dealbreaker for a user demanding a minimum of \$120k.

- **Technique Choice:** We implemented a strict Multi-Stage Funnel to combine the best of both worlds:
 1. **L1 (Candidate Generation):** FAISS retrieves the top 200 semantic matches.
 2. **L2 (Hard Filtering):** Deterministic boolean logic iterates over the L1 pool and eradicates any jobs that violate the user's explicit parameters (Location mismatches, lack of Visa Status sponsorship, Salary below threshold).
 3. **L3 (Weighted Re-Ranking):** The surviving jobs are re-scored using a composite formula to balance semantic depth with exact skill matching:

```
Final Score = (0.50 * FAISS Semantic Distance Score)
              + (0.35 * Explicit Skill Overlap Percentage)
              + (0.15 * Location and Title Exact Match Bonus)
```

3.3 Adaptive Semantic Feedback (Lecture 8)

A static ranking system cannot learn a user's unstated or evolving preferences over the course of a session. If a user consistently ignores or dislikes "Senior Level" roles despite a high semantic match, the

system must adapt.

- **Technique Choice:** We built an Implicit Feedback Loop based on User Interaction. When a user clicks "Pass" on a specific job card, the system extracts the target company and the semantic role tier (e.g., "Senior", "Manager"). It then applies a **Semantic Penalty Array**. Upon the next UI refresh, the Re-Ranker dynamically recalculates and lowers the scores of all structurally similar jobs. Conversely, clicking "Like" applies a localized boost to the cluster surrounding that job. This ensures visible, real-time adaptation to the user's latent intent.

4. Test Personas & Rigorous Evaluation

To empirically evaluate the system's performance, we designed an offline benchmarking suite utilizing four distinct "Edge Case" Personas. We measured ranking quality using **NDCG@10** (Normalized Discounted Cumulative Gain) and **Precision@10** against manually annotated ground truth targets.

4.1 Persona Profiles & Edge Case Behaviors

1. **Aisha (Career Pivoter):** Aisha is transitioning from Marketing to Data Analytics. Her resume lacks standard data titles but possesses the underlying analytical skills. *Edge Case Handled: Dense embeddings successfully solved the vocabulary gap that crippled the BM25 baseline.*
2. **Marcus (New Grad):** Marcus has numerous academic projects but zero industry experience. *Edge Case Handled: The Adaptive Re-ranker successfully learned his preference and penalized Senior/Lead roles after two "Pass" clicks.*
3. **Priya (Experienced Niche):** Priya is highly specialized in a rare, legacy tech stack. *Edge Case Handled: The L3 Multi-Stage logic (specifically the 35% Skill Overlap weight) corrected the embedding model's tendency to over-generalize her niche skills.*
4. **Kenji (International Visa):** Kenji requires immediate H1-B visa sponsorship. Non-sponsoring companies are absolute dealbreakers regardless of how perfectly the skills match. *Edge Case Handled: The L2 Hard Filter reliably eradicated all false-positive matches that the FAISS index initially returned.*

4.2 Offline Benchmark Results Comparison

Persona	Baseline (BM25)	Embedding Only (L5)	Multi-Stage Re-Ranking (L7)
Aisha (Pivoter)	0.00	0.82	0.98
Marcus (New Grad)	0.35	0.70	1.00
Priya (Niche)	0.85	0.75	0.96
Kenji (Visa req.)	0.40	0.65	1.00

Analysis Note: Observe how Priya's embedding-only score dropped slightly below the BM25 baseline. This is a known phenomenon where dense vectors over-generalize rare keywords. Our implementation of the Multi-Stage L7 pipeline successfully mitigated this regression.

4.3 Real-Time Online Evaluation Metrics

Beyond offline benchmarks, the Streamlit UI features a live analytics dashboard that calculates real-time **User Precision (Hit Rate)** and **Top-10 Skill Coverage (Recall)** directly driven by the user's session interactions. This provides a transparent view of the Adaptive Feedback system's effectiveness.

5. System Limitations & Future Architectural Enhancements

While JobPilot demonstrates significant improvements over baseline matching systems and effectively utilizes BAX-423 concepts, several architectural limitations remain:

1. **Embedding Context Window Truncation:** The `all-MiniLM-L6-v2` transformer enforces a strict 256-token limit. Many enterprise job descriptions exceed 1,000 words. Consequently, specific technical requirements located at the bottom of a posting may be truncated before they are mathematically embedded.
 - *Proposed Solution:* Implement a sliding window document chunking strategy with Max-Pooling across chunks, or upgrade to a modern long-context embedding model such as `Nomic-Embed` (which supports an 8k token window).
2. **LLM Extraction Synchronous Latency:** Relying on the cloud-based Gemini API for parsing unstructured PDF resumes introduces a synchronous network bottleneck (typically 2-4 seconds) during user onboarding. This degrades the initial user experience and disrupts the otherwise real-time feel of the application.
 - *Proposed Solution:* Deprecate the cloud LLM dependency for basic onboarding and deploy a quantized, local Named Entity Recognition (NER) model (e.g., GLiNER or a fine-tuned spaCy pipeline) to extract skills and titles instantaneously on the edge.
3. **Feedback Sparsity and the Cold Start Problem:** The adaptive semantic feedback engine performs exceptionally well after 5 or more interactions, but brand-new users face a "cold start" where initial rankings rely solely on zero-shot embeddings.
 - *Proposed Solution:* Implement Cross-User Collaborative Filtering. By persisting session data and analyzing the interaction histories of users with similar demographic and skill profiles, the system can dynamically assign default interaction weights to new users before their very first click.
4. **FAISS Index Volatility:** Currently, the FAISS index is rebuilt entirely in memory. As the SQLite database grows via the streaming Adzuna API, rebuilding the index becomes computationally expensive.
 - *Proposed Solution:* Transition to an incremental IndexIVFFlat FAISS architecture, allowing for batch vector additions without requiring a full re-computation of the vector space.